

BitQuan

A Post-Quantum Proof-of-Work Blockchain
with CRYSTALS-Dilithium5 Signatures

BitQuan Contributors

March 2026

Satoshi Nakamoto’s blockchain architecture established a new model of decentralized trust, enabling value transfer without central authorities. The cryptographic foundation chosen at genesis—elliptic curve signatures such as ECDSA—represented the strongest practical standards of that era. As quantum computing matures, however, Shor’s algorithm transitions from theoretical concern to credible threat: a sufficiently capable quantum computer can derive private keys directly from public keys, undermining the ownership guarantees on which all major blockchain networks depend.

BitQuan exists to ensure that Satoshi’s vision survives the quantum era. Rather than retrofitting post-quantum cryptography onto an existing chain—a process requiring hard forks, user migration, and prolonged vulnerability windows—BitQuan is architected from genesis with CRYSTALS-Dilithium5 (NIST FIPS 204) as its sole signature scheme. The result is a proof-of-work blockchain providing 256-bit classical and quantum security from Block 0, with no migration ever required. This whitepaper details the protocol design, consensus mechanism, and security analysis underpinning this approach.

BitQuan is a proof-of-work blockchain that uses CRYSTALS-Dilithium5 digital signatures, standardized by NIST in FIPS 204, to provide post-quantum security from genesis. Unlike existing blockchains that rely on ECDSA or Ed25519—both vulnerable to Shor’s algorithm on a sufficiently powerful quantum computer—BitQuan achieves 256-bit classical and quantum security without requiring a future migration. This whitepaper describes the protocol design, consensus mechanism, cryptographic foundations, and security analysis of BitQuan.

Contents

1	Introduction	4
1.1	The Quantum Threat	4
1.1.1	Shor’s Algorithm and Blockchain Impact	4
1.1.2	Quantum Computing Timeline	4
1.2	BitQuan’s Approach	5
1.3	Design Principles	5
1.4	Monetary Policy	5
2	Background	5
2.1	Post-Quantum Cryptography	5
2.1.1	Hardness Assumptions	6
2.2	CRYSTALS-Dilithium	6
2.2.1	Dilithium Algorithm Overview	6
2.3	Why Not Hybrid Mode?	7
2.4	BLAKE3 Hash Function	7
3	Protocol Design	7
3.1	Transaction Model	7
3.1.1	Formal UTXO Model	8
3.1.2	Transaction Walkthrough	8
3.2	Transaction Lifecycle	8
3.3	Transaction Structure	9
3.4	Address Format	9
3.5	Block Structure	9
4	Consensus Mechanism	10
4.1	Proof-of-Work	10
4.1.1	Compact Difficulty Encoding	10
4.1.2	Hybrid Mining (Testnet)	10
4.2	ASERT Difficulty Adjustment	10
4.3	BurstGuard	11
4.4	Chain Selection	11
5	Cryptographic Module	11
5.1	Cryptographic Stack	11
5.2	Wallet Security	12
5.2.1	Argon2id KDF Parameters	12
5.2.2	Brute-Force Lockout Schedule	12
5.3	Signature Size Trade-off	13
6	Security Analysis	13
6.1	Post-Quantum Security	13
6.1.1	Security Level Analysis	13
6.1.2	“Harvest Now, Decrypt Later” Resistance	13
6.2	Attacker Success Probability	13
6.3	Classical Security Properties	14
6.4	Implementation Security	14
6.4.1	STRIDE Threat Model Summary	15

7	Network Architecture	15
7.1	P2P Protocol	15
7.1.1	Network Architecture Diagram	15
7.1.2	P2P Message Types	16
7.2	RPC Interface	16
8	Performance	16
8.1	Effective TPS Analysis	17
8.2	Limitations and Trade-offs	17
9	Comparison with Existing Solutions	17
9.1	Security Comparison	18
10	Future Work	18
10.1	Development Roadmap	18
11	Conclusion	18
11.1	Call to Action	19

1 Introduction

In 1994, Peter Shor demonstrated that a sufficiently large quantum computer could break the elliptic curve and discrete logarithm problems that underpin most blockchain signatures [1]. The National Institute of Standards and Technology (NIST) has warned that “migration to post-quantum cryptography must begin now” [2].

Current blockchain systems using ECDSA (Bitcoin, Ethereum) or Ed25519 (Solana, Cardano) face an existential risk: once a quantum computer with approximately 4,000 logical qubits is built, an attacker could derive private keys from public keys, forge signatures, and steal funds.

1.1 The Quantum Threat

1.1.1 Shor’s Algorithm and Blockchain Impact

Shor’s algorithm solves two problems in polynomial time on a quantum computer:

1. **Integer Factorization:** Given $n = p \cdot q$, find p and q in $O((\log n)^3)$ time (classical: sub-exponential).
2. **Discrete Logarithm:** Given g and $h = g^x \pmod{p}$, find x in polynomial time.

For blockchain signatures, the discrete logarithm variant is the direct threat:

- **ECDSA** ($y = x \cdot G$ on secp256k1): Shor’s algorithm can recover private key x from public key y .
- **Ed25519** ($A = s \cdot B$ on Curve25519): Similarly vulnerable via the discrete logarithm problem.
- **RSA** ($n = p \cdot q$, signature via d): Broken via integer factorization.

Critical distinction. A quantum adversary needs only the *public key* to derive the private key. Since blockchain transactions expose public keys, all historical transactions are vulnerable retroactively—an attacker with a quantum computer could forge signatures for any address that has ever made a transaction.

1.1.2 Quantum Computing Timeline

Table 1: Quantum Computing Milestones and Blockchain Risk

Year	Milestone	Blockchain Impact
1994	Shor’s algorithm published	Theoretical threat identified
2016	NIST PQC standardization begins	Preparation phase
2019	Google achieves quantum supremacy (53 qubits)	Proof of concept, not yet practical
2024	NIST publishes FIPS 204 (Dilithium)	PQC standards available
2026–2030	1,000–4,000 logical qubits (est.)	ECDSA/Ed25519 breakable
2030+	Fault-tolerant quantum computers	“Harvest now, decrypt later” becomes real

Note: The qubit counts above refer to *logical* (error-corrected) qubits, not physical qubits. Current error-correction techniques require approximately 1,000–10,000 physical qubits per logical qubit; the practical timeline therefore depends heavily on advances in quantum error correction.

The “harvest now, decrypt later” attack is particularly dangerous for blockchains: an adversary can collect public keys today and derive private keys once a capable quantum computer becomes available.

1.2 BitQuan's Approach

BitQuan addresses this threat by using CRYSTALS-Dilithium5 [3] as its sole signature scheme from genesis:

- **NIST Standardized:** Selected by NIST in 2024 as FIPS 204, providing regulatory confidence.
- **Security Level 5:** 256-bit security against both classical and quantum adversaries.
- **No Migration Needed:** Unlike post-quantum upgrade proposals for existing chains, BitQuan requires no hard fork or user action.
- **50+ Year Horizon:** Designed to remain secure against quantum computers expected through 2075+.

1.3 Design Principles

1. **Security First:** Post-quantum cryptography as the foundation, not an add-on.
2. **Simplicity:** UTXO model with minimal protocol complexity.
3. **Memory Safety:** Implemented in Rust with `#![forbid(unsafe_code)]`.
4. **Transparency:** All transactions are publicly verifiable.

1.4 Monetary Policy

BitQuan follows a deflationary model with tail emission:

- **Max Supply:** 21,000,000 BQ (21 million, pre-tail).
- **Unit:** 1 BQ = 10^{18} qbits (18 decimal places, stored as u128).
- **Block Reward:** Starts at 50 BQ, halves every 210,000 blocks.
- **Tail Emission:** 0.5 BQ per block after 127 halvings, ensuring perpetual mining incentive.
- **Coinbase Maturity:** 100 blocks before a mining reward can be spent.
- **Halving Schedule:** Approximately every 4 years.

The total pre-tail supply in qbits is $21,000,000 \times 10^{18} = 2.1 \times 10^{25}$ qbits.

2 Background

2.1 Post-Quantum Cryptography

Post-quantum cryptography (PQC) refers to algorithms believed to be secure against both classical and quantum computers. NIST's Post-Quantum Cryptography Standardization Process (2016–2024) evaluated 82 submissions and selected CRYSTALS-Dilithium for digital signatures and CRYSTALS-Kyber for key encapsulation.

2.1.1 Hardness Assumptions

PQC algorithms are based on mathematical problems for which no polynomial-time quantum algorithm is known:

Table 2: PQC Problem Families

Problem	PQC Scheme	Use Case
Module-LWE (MLWE)	CRYSTALS-Kyber	Key encapsulation
Module-SIS (MSIS)	CRYSTALS-Dilithium	Digital signatures
Structured Lattices	NTRU, Falcon	Encryption, signatures
Hash-based	SPHINCS+	Stateless signatures
Isogenies	SIKE (broken)	Key exchange
Code-based	Classic McEliece	Key encapsulation
Multivariate	Rainbow (broken)	Signatures

Note: SIKE was broken in 2022 by a classical attack [4], demonstrating the importance of NIST’s thorough evaluation process.

2.2 CRYSTALS-Dilithium

Dilithium is a lattice-based signature scheme based on the Module-Learning with Errors (MLWE) and Module-Small Integer Solution (MSIS) problems:

Table 3: Dilithium Security Levels (NIST FIPS 204)

Variant	Classical	Quantum	Signature Size
Dilithium2	128-bit	128-bit	2,420 bytes
Dilithium3	192-bit	192-bit	3,309 bytes
Dilithium5	256-bit	256-bit	4,627 bytes

BitQuan uses Dilithium5 for the maximum security margin.

2.2.1 Dilithium Algorithm Overview

The Dilithium signature scheme consists of three algorithms: KeyGen, Sign, and Verify.

Key Generation (KeyGen):

1. Generate a random matrix $A \in \mathbb{Z}_q^{k \times l}$ and vectors s_1, s_2 with small entries.
2. Compute $t = A \cdot s_1 + s_2 \pmod{q}$.
3. Public key: (A, t) . Private key: (A, t, s_1, s_2) .

Signing (Sign):

1. Sample a random vector y with Gaussian-like distribution.
2. Compute commitment $w_1 = \text{HighBits}(A \cdot y) \pmod{q}$.
3. Derive challenge $c = H(\text{msg}, w_1)$ via hash function H .
4. Compute response $z = y + c \cdot s_1$ (with rejection sampling).
5. Output signature (c, z, h) where h encodes hint information.

Verification (Verify):

1. Reconstruct w'_1 from (A, c, z, h) .
2. Recompute $c' = H(\text{msg}, w'_1)$.
3. Accept iff $c' = c$ and $\|z\|_\infty, \|A \cdot z - t \cdot c\|_\infty$ are within bounds.

Correctness. Verification succeeds because:

$$A \cdot z - t \cdot c = A(y + c \cdot s_1) - (A \cdot s_1 + s_2) \cdot c = A \cdot y - s_2 \cdot c. \quad (1)$$

The high-bits of this value equal w_1 , so the recomputed challenge matches.

2.3 Why Not Hybrid Mode?

A hybrid approach (ECDSA + Dilithium) provides a fallback if Dilithium is broken. However:

- It adds protocol complexity and increases transaction sizes.
- ECDSA provides no additional security against quantum adversaries.
- If Dilithium5 (NIST Level 5) is broken, all lattice-based cryptography is likely broken, rendering ECDSA irrelevant.
- An optional hybrid mode for interoperability may be added in a future version.

2.4 BLAKE3 Hash Function

BitQuan uses BLAKE3 for address hashing:

- **Speed:** 5–10× faster than SHA-256 on modern hardware.
- **Parallelism:** Supports arbitrary-length parallel hashing.
- **Output:** Extendable output function (XOF) with arbitrary length.
- **Security:** 256-bit, resistant to length-extension attacks.

SHA-256d remains the block-hashing algorithm for proof-of-work, preserving compatibility with the existing SHA-256 mining ecosystem.

3 Protocol Design

3.1 Transaction Model

BitQuan uses the Unspent Transaction Output (UTXO) model, identical to Bitcoin:

- Each transaction consumes one or more UTXOs (inputs) and creates new UTXOs (outputs).
- Double-spending is prevented by verifying that each input UTXO exists and has not been spent.
- Transaction malleability is addressed by signing the serialized transaction without scripts.

3.1.1 Formal UTXO Model

Definition. The UTXO set at height h is:

$$\mathcal{U}_h = \bigcup_{i=0}^h \text{Outputs}(\text{Block}_i) \setminus \bigcup_{i=0}^h \text{Inputs}(\text{Block}_i). \quad (2)$$

Transaction Validity. A transaction τ is valid at height h iff:

1. For each input $(txid_j, idx_j)$: $(txid_j, idx_j) \in \mathcal{U}_h$.
2. $\sum_{\text{out} \in \tau.\text{outputs}} \text{out.value} \leq \sum_{\text{in} \in \tau.\text{inputs}} \text{in.value}.$
3. Each Dilithium5 signature σ_j verifies against the public key in UTXO $(txid_j, idx_j)$.
4. No input UTXO appears more than once (double-spend prevention).

State Transition. The UTXO set evolves as:

$$\mathcal{U}_{h+1} = (\mathcal{U}_h \setminus \text{Inputs}(\tau)) \cup \text{Outputs}(\tau), \quad (3)$$

where τ is the set of transactions in block $h + 1$.

3.1.2 Transaction Walkthrough

Consider a payment from Alice to Bob for 15 BQ. Alice controls a UTXO worth 50 BQ:

1. Alice locates her UTXO: 50 BQ at output 0 of transaction `tx_abc`.
2. She constructs a transaction with:
 - **Input 0:** `prev_txid = tx_abc, prev_index = 0`.
 - **Output 0:** 15 BQ to Bob's address `bq1q...`
 - **Output 1:** $34.9999\bar{9}$ BQ to Alice's change address.
3. The difference ($\approx 10^9$ qbits) is the transaction fee.
4. Alice signs the transaction hash with her Dilithium5 private key, producing a 4,627-byte signature.
5. The transaction ($\sim 8,600$ weight units) is broadcast to the mempool.
6. A miner includes the transaction in the next block.

The total on-chain size is approximately 8.6 KB, versus ~ 225 bytes for an equivalent Bitcoin transaction—a $38\times$ increase attributable to the Dilithium5 signature.

3.2 Transaction Lifecycle

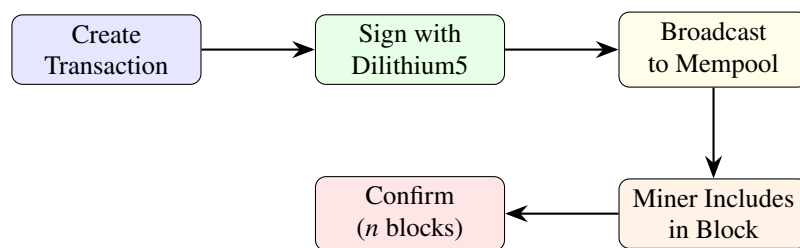


Figure 1: Transaction lifecycle from creation to confirmation.

3.3 Transaction Structure

Listing 1: Transaction and UTXO structures (simplified Rust).

```

1 struct Transaction {
2     version:    u32,
3     inputs:     Vec<TxIn>,
4     outputs:    Vec<TxOut>,
5     lock_time:  u64,
6     signatures: Vec<DilithiumSignature>,
7 }
8
9 struct TxIn {
10     prev_txid: [u8; 32], // Previous transaction hash (BLAKE3)
11     prev_index: u32,     // Output index in previous tx
12     script:    Vec<u8>, // Unlocking script
13 }
14
15 struct TxOut {
16     value: u128, // Amount in qbits (1 BQ = 10^18 qbits)
17     script: Vec<u8>, // Locking script (P2PKH or P2SH)
18 }

```

3.4 Address Format

BitQuan addresses use bech32 encoding with the human-readable prefix bq:

Table 4: Address Types

Type	Prefix	Description
P2PKH	bq1q	Pay-to-Public-Key-Hash (BLAKE3 of Dilithium5 public key)
P2SH	bq1p	Pay-to-Script-Hash (planned)

Since BitQuan uses only Dilithium5 from genesis, the P2PKH type always hashes a Dilithium5 public key; there is no classical ECDSA variant. Address derivation:

1. Generate Dilithium5 keypair (pk, sk) .
2. Hash: $h = \text{BLAKE3}(pk)$.
3. Encode: address = bech32(bq, 0, h).

A BitQuan address is longer than a Bitcoin address because BLAKE3 must cover the 1,952-byte Dilithium5 public key versus Bitcoin's 33-byte compressed ECDSA key.

3.5 Block Structure

Listing 2: Block and BlockHeader structures (simplified Rust).

```

1 struct Block {
2     header:    BlockHeader,
3     transactions: Vec<Transaction>,
4 }
5
6 struct BlockHeader {
7     version:    u32,
8     prev_hash:  [u8; 32], // SHA-256d of previous header

```

```

9   merkle_root: [u8; 32], // Merkle root of transactions
10  timestamp:   u64,      // Unix timestamp
11  difficulty:  Compact,  // Compact difficulty target
12  nonce:       u64,      // Proof-of-work nonce
13  coinbase_tx: Transaction,
14 }

```

4 Consensus Mechanism

4.1 Proof-of-Work

BitQuan uses SHA-256d (double SHA-256) as the primary mining algorithm, identical to Bitcoin. A block is valid if:

$$\text{SHA256d}(\text{block_header}) < \text{target}, \quad (4)$$

where *target* is derived from the compact difficulty bits.

4.1.1 Compact Difficulty Encoding

Difficulty is encoded as a 32-bit integer using Bitcoin's compact format:

$$\text{target} = \text{coefficient} \cdot 256^{(\text{exponent}-3)}, \quad (5)$$

where $\text{exponent} = \lfloor \text{bits}/2^{24} \rfloor$ and $\text{coefficient} = \text{bits} \bmod 2^{24}$. The difficulty D is:

$$D = \frac{\text{target}_{\text{genesis}}}{\text{target}_{\text{current}}}. \quad (6)$$

4.1.2 Hybrid Mining (Testnet)

On testnet and devnet, BitQuan supports hybrid mining with multiple algorithms:

Table 5: Mining Algorithms by Network

Network	SHA-256d	RandomX	Ethash	Hybrid
Mainnet	Yes	No	No	Yes (block 10,000+)
Testnet	Yes	Yes	Yes	Yes (block 1,000+)
Devnet	Yes	Yes	Yes	Yes
Regtest	Yes	Yes	Yes	Yes

Hybrid mode enables inclusive participation across hardware types. On mainnet, it activates at block 10,000 following an initial SHA-256d-only bootstrapping phase; SHA-256d remains the canonical algorithm for security.

4.2 ASERT Difficulty Adjustment

BitQuan uses Aserti3-2D (ASERT) for difficulty adjustment, providing smooth transitions without the staircase effect of Bitcoin's per-epoch DAA [5]:

$$D_{\text{new}} = D_{\text{anchor}} \cdot 2^{\frac{t_{\text{actual}} - t_{\text{expected}}}{T_{1/2}}}, \quad (7)$$

where:

- D_{anchor} is the difficulty of the reference (anchor) block.
- t_{actual} is the actual elapsed time since the anchor block.
- $t_{\text{expected}} = \Delta h \cdot T_{\text{block}}$ is the expected elapsed time for Δh blocks at the target interval.
- $T_{1/2} = 172,800$ s (2 days) is the half-life parameter.

Properties:

- Single-parameter tuning (only $T_{1/2}$).
- Smooth transitions: no epoch-boundary staircase effect.
- Exponential averaging: stable under hash-rate oscillations.
- Self-correcting: over-fast blocks increase difficulty proportionally.

Bitcoin adjusts difficulty every 2,016 blocks (≈ 2 weeks) with a discrete step function. ASERT adjusts continuously—every block—using an exponential function that naturally smooths perturbations.

4.3 BurstGuard

BurstGuard is a spike-protection mechanism that prevents difficulty manipulation through burst mining:

1. Maintain a sliding window W of the last 100 block timestamps.
2. Compute observed rate: $r_{\text{obs}} = |W| / (W_{\text{last}} - W_{\text{first}})$.
3. Define expected rate: $r_{\text{exp}} = 1 / T_{\text{target}}$ where $T_{\text{target}} = 600$ s.
4. If $r_{\text{obs}} > \theta \cdot r_{\text{exp}}$ ($\theta = 1.5$, spike threshold):
 - (a) Apply multiplier: $D_{\text{burst}} = D \cdot (r_{\text{obs}} / (\theta \cdot r_{\text{exp}}))$.
 - (b) Cap: $D_{\text{burst}} \leq 16 \cdot D$ (max 16 $\times$).

Attack prevented. A miner submitting 50 blocks in rapid succession cannot depress long-term difficulty, because BurstGuard applies a proportional penalty to the observed hash rate.

4.4 Chain Selection

The chain with the highest cumulative difficulty is canonical. In case of a tie, the chain with the numerically lower block hash wins.

5 Cryptographic Module

5.1 Cryptographic Stack

Table 6: BitQuan Cryptographic Stack

Component	Algorithm	Standard
Signatures	CRYSTALS-Dilithium5	NIST FIPS 204
Key Derivation	Argon2id (256 MiB)	RFC 9106
Symmetric Encryption	AES-256-GCM	NIST SP 800-38D
Block Hashing (PoW)	SHA-256d	NIST FIPS 180-4
Address Hashing	BLAKE3	BLAKE3 spec
Mnemonic Seed	BIP-39	BIP-39
Random Numbers	OS CSPRNG	getrandom crate

5.2 Wallet Security

Wallet private keys are protected by multiple layers:

1. **Argon2id KDF**: 256 MiB memory, 3 time iterations, parallelism 1—makes brute-force economically infeasible.
2. **AES-256-GCM**: Authenticated encryption prevents tampering.
3. **Memory Locking**: `mlock()` prevents keys from being swapped to disk.
4. **Zeroization**: `zeroize` crate ensures keys are erased on drop.
5. **Constant-Time**: `subtle::ConstantTimeEq` prevents timing-side-channel attacks.
6. **Brute-Force Protection**: Exponential backoff with progressive lockout.

5.2.1 Argon2id KDF Parameters

Table 7: KDF Parameter Comparison

Parameter	OWASP Min	BitQuan	Bitcoin Core
Algorithm	Argon2id	Argon2id	SHA-512
Memory	64 MiB	256 MiB	N/A
Time Cost	2	3	1 iteration
Parallelism	4	1	N/A
Output	32 B	32 B	32 B
KDF Time	~0.5 s	~1.5 s	~0.1 s

At 256 MiB per attempt and 1.5 s per iteration, online password-guessing attacks are economically infeasible even with large GPU clusters.

5.2.2 Brute-Force Lockout Schedule

Table 8: Progressive Lockout Schedule

Failed Attempts	Delay	Lockout
1–3	0 s	No
4–5	1 s	No
6–7	2 s	No
8–9	4 s	No
10–14	8 s	No
15–19	16 s	No
20+	30 s + 60 s lockout	Yes

Delay formula: $\text{delay}(a) = \min(2^{\lfloor (a-3)/2 \rfloor}, 30)$ s, where a is the attempt count. Lockout resets on successful authentication.

5.3 Signature Size Trade-off

Dilithium5 signatures are approximately 63× larger than ECDSA signatures by total bytes:

Table 9: Signature Size Comparison

	ECDSA	Ed25519	Dilithium5
Public Key	33 B	32 B	1,952 B
Signature	72 B	64 B	4,627 B
Total	105 B	96 B	6,579 B

Mitigation strategies include a 4 MW block weight limit, future signature aggregation (batch Dilithium), and off-chain payment channels.

6 Security Analysis

6.1 Post-Quantum Security

BitQuan’s security relies on the hardness of Module-LWE and Module-SIS—problems for which no polynomial-time quantum algorithm is known. NIST’s eight-year standardization process provides confidence in this assumption.

6.1.1 Security Level Analysis

Dilithium5 provides NIST Security Level 5:

- Classical security: 2^{256} operations to break (equivalent to AES-256).
- Quantum security: 2^{256} operations on a quantum computer.
- Exceeds the 128-bit security of ECDSA by a factor of 2^{128} .

Implication. A quantum computer capable of breaking ECDSA in seconds would still require 2^{256} operations against BitQuan—computationally infeasible.

6.1.2 “Harvest Now, Decrypt Later” Resistance

An adversary collecting BitQuan public keys today cannot derive private keys later because:

1. Dilithium’s security reduces to worst-case lattice problems with no known quantum speedup.
2. NIST’s analysis considered all known quantum algorithms, including Grover’s algorithm.
3. The 256-bit margin provides headroom for future algorithmic improvements.

6.2 Attacker Success Probability

Following the analysis in the Bitcoin whitepaper [?], let p be the probability an honest node finds the next block and $q = 1 - p$ the attacker’s probability. The probability the attacker catches up from z blocks behind is:

$$P(\text{catch up}) = \begin{cases} 1 & \text{if } p \leq q, \\ \left(\frac{q}{p}\right)^z & \text{if } p > q. \end{cases} \quad (8)$$

For a recipient waiting z confirmations, the attacker's progress follows a Poisson distribution with mean $\lambda = z \cdot q/p$:

$$P = 1 - \sum_{k=0}^z \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - \left(\frac{q}{p} \right)^{z-k} \right). \quad (9)$$

Table 10: Attacker Success Probability for Varying q and Confirmation Count z

q	$z = 1$	$z = 6$	$z = 12$	$z = 24$
0.10	0.2046	0.0002	$< 10^{-9}$	$< 10^{-18}$
0.15	0.3358	0.0012	$< 10^{-7}$	$< 10^{-14}$
0.25	0.5000	0.0156	0.0002	$< 10^{-8}$
0.30	0.5882	0.0467	0.0022	$< 10^{-5}$
0.45	0.8182	0.3512	0.1234	0.0152

Quantum advantage on proof-of-work. Grover's algorithm provides a quadratic speedup on SHA-256d, effectively doubling an attacker's hash rate for the same energy budget. However, a quantum attacker *cannot* forge Dilithium5 signatures—they can only attempt to outpace honest miners. The confirmation analysis above applies equally to BitQuan and Bitcoin; recommended confirmation counts for a given transaction value are the same.

6.3 Classical Security Properties

- **Double-Spend Prevention:** UTXO model with hash-indexed lookup.
- **Transaction Immutability:** Merkle-tree commitment in every block header.
- **Sybil Resistance:** Proof-of-Work requires real computational effort.
- **Eclipse Resistance:** Multiple bootstrap nodes and enforced outbound peer diversity.

6.4 Implementation Security

- **No Unsafe Code:** `#![forbid(unsafe_code)]` at workspace level.
- **Memory Safety:** Rust's ownership system eliminates buffer overflows, use-after-free, and data races.
- **Strict Linting:** Clippy with `-D warnings`—all warnings are errors.
- **Fuzzing:** 24+ hour campaigns with zero crashes.
- **Dependency Auditing:** `cargo audit` and `cargo deny` in CI.

6.4.1 STRIDE Threat Model Summary

Table 11: STRIDE Analysis Summary

Category	Primary Threat	Mitigation
Spoofing	Fake peer identity	Handshake protocol, ban scoring
Tampering	Malicious block injection	Full validation, Merkle tree
Repudiation	Denying a transaction	Dilithium5 undeniable signatures
Info Disclosure	Private key leakage	mlock, zeroize, log policy
Denial of Service	Mempool spam	Size limits, min fee, eviction
Elevation	RPC auth bypass	JWT validation, rate limiting

7 Network Architecture

7.1 P2P Protocol

BitQuan uses a gossip protocol for block and transaction propagation:

1. Nodes connect to bootstrap peers for initial peer discovery.
2. New blocks and transactions are broadcast to all connected peers.
3. Peers are scored: misbehavior raises a ban score; exceeding the threshold triggers disconnection.
4. Rate limiting and message-size caps prevent denial-of-service attacks.

7.1.1 Network Architecture Diagram

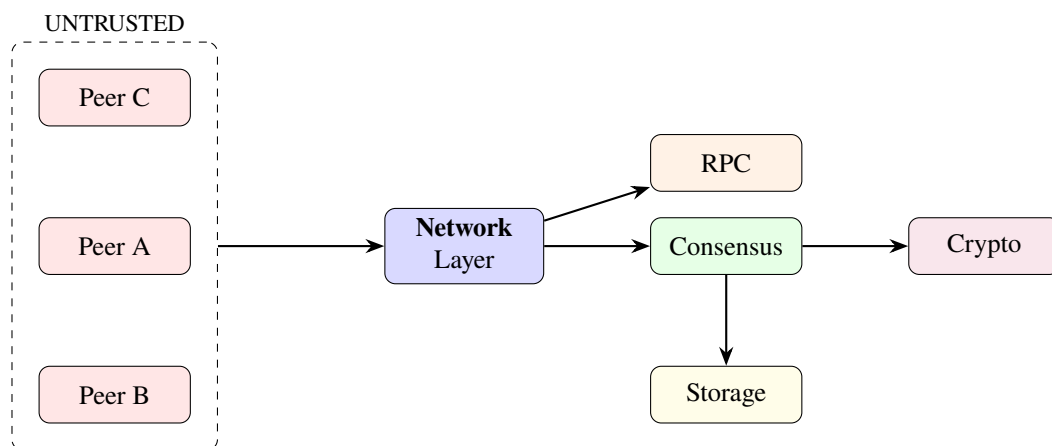


Figure 2: Node architecture with trust boundary.

7.1.2 P2P Message Types

Table 12: P2P Message Types

Message	Description
version	Protocol version negotiation
verack	Version acknowledgment
addr	Peer address exchange
inv	Inventory: announce new blocks/transactions
getdata	Request specific blocks/transactions
block	Full block transmission
tx	Transaction broadcast
ping/pong	Keep-alive
reject	Error reporting with reason codes

7.2 RPC Interface

The JSON-RPC API provides programmatic node access:

- **Blockchain:** getblockcount, getblockchaininfo, getblockhash.
- **Mining:** getblocktemplate, submitblock, getmininginfo.
- **Transactions:** gettransaction, submittransaction, sendtoaddress.
- **Network:** getnetworkstatus, sync.
- **Pool:** getpoolstats, getminerstats, createpayout.

Authentication uses JWT tokens signed with HMAC-SHA256.

8 Performance

Table 13: Key Performance Metrics

Operation	Time
Dilithium5 Key Generation	~1.2 ms
Dilithium5 Sign	~0.8 ms
Dilithium5 Verify	~0.5 ms
ECDSA Sign (comparison)	~0.05 ms
ECDSA Verify (comparison)	~0.1 ms
Argon2id KDF (256 MiB)	~1.5 s
AES-256-GCM Encrypt (1 KB)	~0.005 ms
Block Validation (100 tx)	~150 ms
Block Validation (1,000 tx)	~1.5 s
UTXO Lookup (indexed)	~0.01 ms
Binary Size (stripped)	~5 MB
Build Time (release)	~3 min

8.1 Effective TPS Analysis

With a 4 MW block-weight limit, a 10-minute target block time, and Bitcoin-style weight accounting ($weight = base_bytes \times 4 + witness_bytes \times 1$):

- A simple 1-input/1-output transaction: ~ 500 B base + $\sim 6,600$ B witness = $\sim 8,600$ weight units.
- Max transactions per block: $\approx 4,000,000 / 8,600 \approx 465$.
- Theoretical TPS: $\approx 465 / 600 \approx 0.78$ TPS.

Future signature aggregation (batch Dilithium) could reduce per-transaction overhead by 5–10×, bringing TPS to ~ 4 –8 TPS.

8.2 Limitations and Trade-offs

Low on-chain TPS. At ~ 0.78 TPS, BitQuan is not suited to high-frequency on-chain transactions. Payment channels and layer-2 solutions are the intended scaling path.

Large transaction sizes. A simple payment is ~ 8.6 KB versus ~ 225 bytes for Bitcoin, increasing bandwidth and storage requirements for full nodes.

Blockchain growth. At ~ 465 tx per block, annual chain growth is roughly 200 GB, compared to ~ 50 GB for Bitcoin at current usage.

Signature verification cost. Dilithium5 verification (~ 0.5 ms) is $5\times$ slower than ECDSA (~ 0.1 ms).

No privacy by default. All transactions are transparent. Confidential transaction support is planned for a future protocol version.

These trade-offs are the accepted cost of 256-bit quantum resistance from genesis. Security is the primary design objective; all other properties are subordinate to it.

9 Comparison with Existing Solutions

Table 14: Feature Comparison

	BitQuan	Bitcoin	Monero	Kaspa
Signatures	Dilithium5	ECDSA	Ed25519	ECDSA
Quantum Safe	Yes	No	No	No
UTXO Model	Yes	Yes	No	Yes
Block Time	~ 10 min	~ 10 min	~ 2 min	~ 1 s
Difficulty	ASERT+BurstGuard	DAA (2,016 blk)	LWMA	BGRT
Language	Rust	C++	C++	Go/Rust
Max Supply	21M + tail	21M	∞ (tail at 18.4M)	Unlimited
Halving	210K blocks	210K blocks	Smooth	No halving
Privacy	Transparent	Transparent	Ring CT	Transparent
Smart Contracts	No	Script	No	No
Memory Safety	No unsafe	Manual	Manual	Partial

9.1 Security Comparison

Table 15: Security Model Comparison

	BitQuan	Bitcoin	Monero	Kaspa
Signature Security	256-bit (PQ)	128-bit	128-bit	128-bit
Wallet KDF	Argon2id (256 MiB)	SHA-512	N/A	N/A
Wallet Encryption	AES-256-GCM	AES-256-CBC	N/A	N/A
Memory Safety	Rust forbid unsafe	C++ manual	C++ manual	Partial
External Audit	Pending	Multiple	Multiple	Quarkslab

10 Future Work

1. **Signature Aggregation.** Batch Dilithium verification to amortize per-signature overhead; the BDN scheme could reduce n signatures to a single verification, bringing TPS closer to Bitcoin levels.
2. **Hybrid Mode.** Optional ECDSA+Dilithium for interoperability, allowing trustless cross-chain verification by ECDSA-only chains.
3. **Payment Channels.** Lightning-style bidirectional channels using Dilithium5 funding signatures and HTLC-like contracts for routing.
4. **Minimal Script.** A programmable transaction language that preserves post-quantum security—no opcodes relying on classical hardness assumptions.
5. **Cross-Chain Bridges.** Trustless bridges to Bitcoin and Ethereum via HTLCs adapted for Dilithium5 signatures.
6. **Confidential Transactions.** An optional privacy layer using homomorphic commitments or zero-knowledge proofs.

10.1 Development Roadmap

Table 16: Development Roadmap

Phase	Timeline	Milestones
Alpha	Q2 2026	Public testnet launch, Docker setup
Beta	Q3 2026	Third-party security audit, bug bounty
RC	Q4 2026	Audit remediation, stress testing
v1.0	Q1 2027	Mainnet launch
v1.1	Q2 2027	Signature aggregation, TPS improvement
v2.0	Q4 2027	Payment channels, hybrid mode

11 Conclusion

BitQuan demonstrates that post-quantum blockchain security is achievable today using NIST-standardized cryptography. By adopting CRYSTALS-Dilithium5 at genesis, BitQuan eliminates the risky migration that existing chains will face as quantum computers mature. The protocol inherits Bitcoin’s proven UTXO model and proof-of-work consensus while introducing two novel contributions:

1. **BurstGuard:** A difficulty spike-protection mechanism that prevents burst mining attacks not addressed by standard DAA algorithms.
2. **Memory-Safe Implementation:** A Rust codebase with `#![forbid(unsafe_code)]`, eliminating whole classes of vulnerabilities that have caused critical bugs in C++ blockchain implementations.

The trade-offs—larger signatures, lower effective TPS, higher storage requirements—are an accepted cost for 256-bit quantum resistance designed to remain secure through 2075 and beyond.

11.1 Call to Action

BitQuan is open-source and welcomes contributions from the cryptography, blockchain, and security communities. The project is in alpha phase; a public testnet is planned for Q2 2026. Areas of particular interest:

- Cryptographic review of the Dilithium5 integration and aggregation proposals.
- Formal verification of the consensus rules and UTXO state machine.
- Security auditing of the wallet, P2P networking, and RPC layers.
- Benchmarking and optimization of the signature verification pipeline.

Repository: <https://github.com/AlphaB135/BitQuan>

References

- [1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.
- [2] National Institute of Standards and Technology, “Post-quantum cryptography standardization.” <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024.
- [3] National Institute of Standards and Technology, “FIPS 204: Module-lattice-based digital signature standard.” <https://csrc.nist.gov/pubs/fips/204>, 2024.
- [4] W. Castryck and T. Decru, “An efficient key recovery attack on SIDH,” in *Advances in Cryptology – EUROCRYPT 2023*, Springer, 2023.
- [5] M. Degraaf, J. Toomim, and T. van der Wansem, “Aserti3-2D: Dynamic difficulty adjustment for Bitcoin Cash.” <https://github.com/bitcoincashorg/bitcoincash-upgrade-specifications/blob/master/0003-aserti3-2d.md>, 2021.